TECNICAS DE PROGRAMACION ORIENTADA A OBJETOS			
DOCENTE	ERIC GUSTAVO CORONEL CASTILLO	CICLO	V
EVALUACIÓN	T1	FECHA	Semana 6

PROGRAMACION EN CAPAS Y EL ENFOQUE DE SERVICIOS

INDICACIONES

La Evaluación T1 es una actividad de carácter grupal que integra los conceptos fundamentales del paradigma de programación en capas y el enfoque de servicios. A continuación, se detallan las indicaciones que el equipo debe seguir de manera rigurosa para garantizar la validez y calidad de la entrega.

Organización del equipo

- La evaluación T1 se resuelve en grupo, conforme a los grupos ya formados en la plataforma Blackboard.
- Un único integrante del grupo es el responsable de realizar la entrega completa en la sección correspondiente de la plataforma.
- Todos los integrantes deben participar activamente en el desarrollo de los proyectos, en la elaboración del informe académico y el video explicativo.

Estructura de la carpeta de trabajo

Cree una carpeta de trabajo con el siguiente formato de nombre:

TPOO_T1_GPO#

Donde GPO# representa el número de su grupo. Por ejemplo: **TPOO_T1_GPO1**, **TPOO_T1_GPO2**, etc. Dentro de esta carpeta deberán ubicarse los proyectos desarrollados y el informe académico.

Contenido de la entrega

1. Los proyectos Java desarrollados bajo el paradigma de programación en capas.
2. El informe académico elaborado según la plantilla oficial del curso.
3. Un video explicativo publicado en YouTube (véase la sección de criterios de evaluación).

Formato y plazos de entrega

- Empaquete su carpeta de trabajo en un archivo comprimido en formato ZIP o RAR.
- Suba únicamente el archivo empaquetado en la sección correspondiente de la plataforma Blackboard.
- Es responsabilidad del equipo asegurar que tanto el archivo comprimido como el enlace al video sean entregados dentro de los plazos establecidos.
- No se aceptarán entregas fuera de plazo.

Nota importante: Todos los proyectos deben resolverse estrictamente bajo el concepto de programación en capas y el enfoque de servicios, tal como se ha desarrollado en las sesiones del curso. No se aceptarán soluciones que no respeten esta arquitectura.

INFORME ACADEMICO

El informe académico es el documento formal de comunicación entre el equipo de estudiantes y el docente. Es el resultado de una investigación bibliográfica y de la experimentación práctica realizada durante el desarrollo de los proyectos. Debe redactarse en tercera persona o en primera persona del plural, garantizando coherencia, progresión lógica y ausencia de contradicciones.

La estructura obligatoria del informe académico es la siguiente:

#	Sección	Descripción y contenido esperado
1	**Carátula**	Incluye: nombre de la institución, título del trabajo, nombres de los autores, especialidad, nombre del docente, fecha y año académico.
2	**Índice**	Esquema completo del trabajo con numeración de páginas. Debe tener al menos 3 niveles de profundidad.
3	**Resumen**	Síntesis del informe: justificación, objetivo general, principales resultados y conclusiones. Redacción concisa y precisa.
4	**Introducción**	Contextualiza al lector sobre el tema. Presenta el problema, el enfoque adoptado y los antecedentes relevantes. No desarrolla ideas centrales ni incluye opiniones personales. Toda definición o concepto debe citarse en formato APA.
5	**Fundamento teórico**	Marco conceptual que sustenta los proyectos. Cada concepto o principio debe estar respaldado por citas parentéticas en formato APA. Fuentes válidas: libros y artículos científicos.
6	**Desarrollo del tema**	Sección más importante del informe. Presenta las evidencias del desarrollo de cada proyecto: diagramas de clases, código fuente con explicación, capturas de pantalla de la interfaz y resultados de ejecución. Se organiza en subsecciones por proyecto.

7	**Conclusiones**	Cierra el informe destacando los hallazgos más importantes. No resume el trabajo ni añade información nueva. Integra, refuerza y generaliza los resultados obtenidos.
8	**Lecciones aprendidas**	Mínimo 3 lecciones aprendidas, fundamentadas en la reflexión crítica sobre el proceso de desarrollo. Deben ser específicas y verificables.
9	**Bibliografía**	Mínimo 6 referencias en formato APA (última edición). Solo se aceptan libros y artículos científicos.
10	**Apéndice / Anexos**	Material complementario e ilustrativo (opcional). Incluye datos, diagramas adicionales o documentos de soporte.

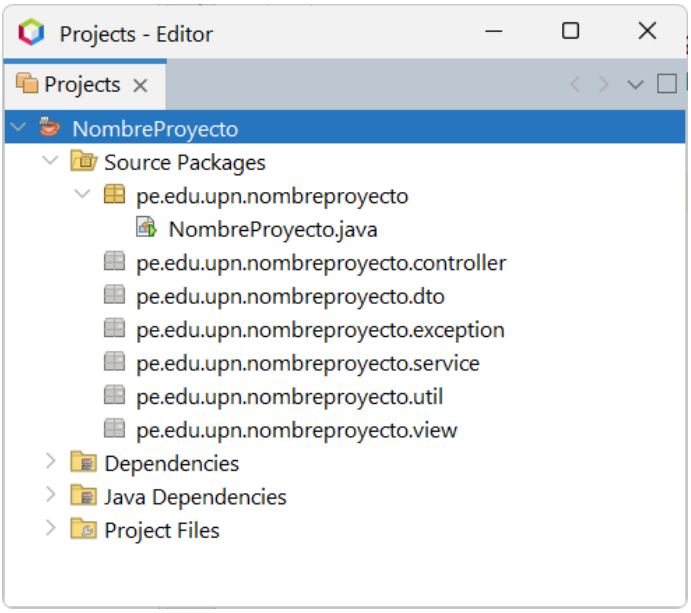
Desarrollo del tema — Evidencias por proyecto

La sección «Desarrollo del tema» es la parte más importante del informe académico. En ella se documentan de manera detallada y ordenada las evidencias que demuestran la correcta implementación de cada proyecto. Para cada proyecto desarrollado, esta sección debe incluir los siguientes elementos:

N°	Elemento de evidencia	Descripción de lo que debe incluir
1	**Descripción del problema**	Explicación clara y concisa del problema que resuelve el proyecto, con su respectivo contexto.
2	**Diagrama de clases**	Diagrama UML que muestre las clases, sus atributos, métodos y las relaciones entre ellas, organizado por capas (service, DTO, controller, view).
3	**Código fuente por capa**	Fragmentos representativos del código Java de cada capa del proyecto (service, controller, view), con una breve explicación de su lógica.
4	**Prueba de servicios**	Evidencia de la ejecución y validación de los métodos de la capa service (resultado de pruebas funcionales).
5	**Capturas de la interfaz**	Capturas de pantalla de la interfaz gráfica (capa view) en funcionamiento, mostrando diferentes escenarios de uso.
6	**Análisis de resultados**	Interpretación de los resultados obtenidos, contrastando con el fundamento matemático o teórico correspondiente.

ESTRUCTURA GENERAL DE PROYECTO JAVA

Todos los proyectos deben seguir la estructura de paquetes que se muestra a continuación. El nombre del paquete raíz sigue la convención de la institución y el nombre del proyecto.



Paquete / Artefacto	Responsabilidad
pe.edu.upn.nombreproyecto	Paquete raíz de la aplicación.
pe.edu.upn.nombreproyecto.NombreProyecto.java	Clase principal (main). Punto de entrada de la aplicación.
pe.edu.upn.nombreproyecto.controller	Clases controladoras que coordinan la lógica entre la vista y el servicio.
pe.edu.upn.nombreproyecto.dto	Objetos de Transferencia de Datos (Data Transfer Objects).
pe.edu.upn.nombreproyecto.exception	Clases para el manejo de excepciones personalizadas.
pe.edu.upn.nombreproyecto.service	Capa de lógica de negocio. Aquí reside el procesamiento principal de cada proyecto.
pe.edu.upn.nombreproyecto.util	Clases utilitarias y métodos auxiliares de uso general.
pe.edu.upn.nombreproyecto.view	Capa de presentación. Gestiona la interfaz gráfica de usuario (GUI).

PROYECTO 1: Clasificación de un número

Desarrollar un programa en Java que permita analizar un número entero positivo N, identificar sus divisores propios y clasificarlo matemáticamente.

Requerimientos funcionales:

1. Encontrar y listar todos los divisores propios de N, excluyendo al propio N.
2. Calcular S(N): la suma de los divisores propios encontrados.
3. Clasificar N según el fundamento matemático descrito a continuación.

Fundamento matemático

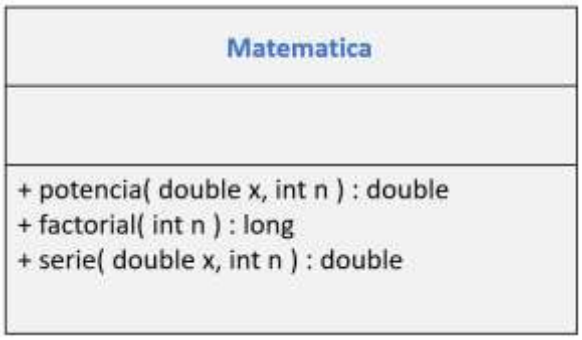
Clasificación	Condición	Ejemplo
Número perfecto	S(N) = N	6 → (1+2+3 = 6) 28 → (1+2+4+7+14 = 28)
Número abundante	S(N) > N	12 → (1+2+3+4+6 = 16 > 12)
Número deficiente	S(N) < N	8 → (1+2+4 = 7 < 8)

Requerimientos de implementación:

- El programa debe implementarse en Java con arquitectura en capas: service, controller y view.
- La lógica de clasificación debe residir completamente en la capa service.
- La interfaz de usuario (capa view) debe ser intuitiva y brindar una experiencia de usuario clara y amigable.

PROYECTO 2: Evaluar Sumatoria

Desarrollar la siguiente clase (sin utilizar la clase Math u otra librería):

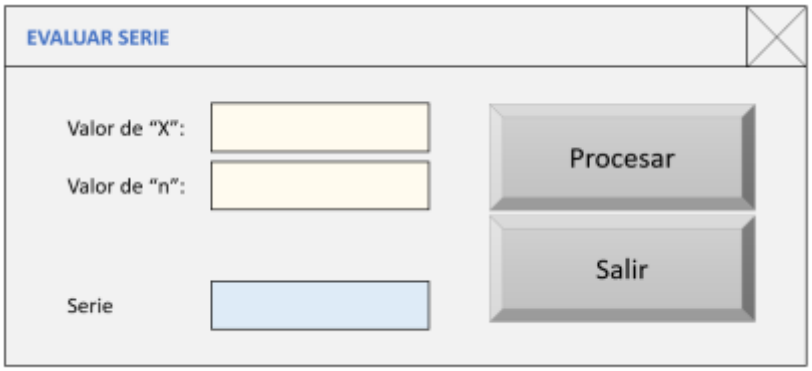


Donde:

- **potencia**: Este método permite elevar el valor de "x" a la potencia "n". El valor de “n” debe ser un numero natural (enteros positivos).
- **factorial**: Este método permite calcular el factorial de "n".
- **serie**: Este método permite evaluar la siguiente serie:

$$serie = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots + \frac{x^{2n-1}}{(2n-1)!}$$

Desarrollar la siguiente interfaz de usuario para evaluar la serie:



Requerimientos de implementación:

- La clase Matematica debe implementarse en la capa service del proyecto.
- La interfaz de usuario debe permitir ingresar los valores de x y n, mostrar el resultado de la serie y tener un botón de salida.
- No está permitido el uso de la clase Math ni de librerías matemáticas externas.
- Se deben validar los valores de entrada (n debe ser entero positivo).

RUBRICA DE EVALUACIÓN

CRITERIO	PESO (%)	INNOVADOR (5) Dominio completo	SÓLIDO (4) Cumple con solidez	LOGRADO (3) Cumple lo esencial	EN DESARROLLO (2) Cumple parcialmente	INICIAL (1) No cumple
PROYECTO 1 **Clasificación de un Número**	**20%**	▪ **Service:** calcula divisores propios, S(N) y clasifica como perfecto, abundante o deficiente. Maneja con excepciones personalizadas entradas inválidas (N ≤ 0). ▪ **Pruebas:** verifica los tres tipos de clasificación con valores representativos e incluye prueba ante entrada inválida. ▪ **View:** muestra divisores, S(N) y clasificación; valida la entrada antes de invocar el servicio y presenta mensajes de error claros.	▪ **Service:** calcula divisores, S(N) y clasifica correctamente. El manejo de excepciones existe, pero no cubre todos los casos inválidos. ▪ **Pruebas:** verifica los tres tipos con resultados correctos, sin probar casos límite. ▪ **View:** muestra clasificación y S(N); la validación de entrada es básica o incompleta.	▪ **Service:** la clasificación funciona, pero hay lógica de presentación mezclada en la capa de servicio o viceversa. ▪ **Pruebas:** prueba solo uno o dos tipos de clasificación; no se consideran casos inválidos. ▪ **View:** funciona, pero omite mostrar la lista de divisores o carece de validación de entrada.	▪ **Service:** la lógica de clasificación es incorrecta para alguno de los tres tipos, o no existe capa service diferenciada. ▪ **Pruebas:** existe al menos una prueba, pero el resultado es incorrecto o insuficiente. ▪ **View:** tiene componentes visuales, pero no está conectada al servicio o produce resultados incorrectos.	▪ **Service:** no existe capa service; toda la lógica está en la vista o el programa no compila. ▪ **Pruebas:** no se presentan pruebas. ▪ **View:** no existe interfaz o está vacía.
PROYECTO 2 **Evaluación de Sumatoria**	**20%**	▪ **Service:** potencia(), factorial() y serie() producen resultados correctos sin usar Math ni librerías externas. La serie evalúa términos con signo alterno correctamente. Se manejan excepciones ante n ≤ 0. ▪ **Pruebas:** verifica los tres métodos con distintos valores de x y n; incluye prueba ante entrada inválida. ▪ **View:** permite ingresar x y n, muestra el resultado con precisión decimal, valida que n sea entero positivo y tiene botón Salir funcional.	▪ **Service:** los tres métodos producen resultados correctos para valores estándar. El manejo de excepciones existe, pero no cubre todos los casos inválidos. ▪ **Pruebas:** verifica la serie con al menos dos pares (x, n) correctos, sin probar los métodos auxiliares por separado. ▪ **View:** muestra el resultado; la validación de n es básica o incompleta.	▪ **Service:** los métodos funcionan para valores pequeños de n, pero la acumulación de términos falla para valores grandes; o se usa alguna función de Math de forma puntual. ▪ **Pruebas:** solo se prueba un par (x, n) con resultado correcto. ▪ **View:** tiene los campos de entrada y resultado, pero la validación o presentación son deficientes.	▪ **Service:** uno o más métodos de la clase Matematica están ausentes o producen resultados incorrectos; o la serie no acumula los términos con el signo correcto. ▪ **Pruebas:** existe al menos una prueba, pero el resultado es incorrecto. ▪ **View:** existe, pero no invoca correctamente los métodos del servicio.	▪ **Service:** la clase Matematica no existe o ninguno de sus métodos está implementado; o el programa no compila. ▪ **Pruebas:** no se presentan pruebas. ▪ **View:** no existe interfaz o está vacía.

INFORME ACADÉMICO	**20%**	▪ Contiene las 10 secciones obligatorias completas y bien desarrolladas. ▪ El desarrollo del tema presenta evidencias de ambos proyectos: diagrama de clases, fragmentos de código explicados, capturas de interfaz y análisis de resultados. ▪ Mínimo 5 referencias en formato APA correcto (libros o artículos científicos); todos los conceptos del fundamento teórico tienen cita parentética. ▪ Redacción coherente, sin contradicciones, en tercera persona o primera persona del plural. ▪ Al menos 3 lecciones aprendidas fundamentadas en reflexión crítica del proceso.	▪ Contiene las secciones obligatorias; alguna sección menor está incompleta (ej. menos de 3 lecciones aprendidas o apéndice ausente). ▪ El desarrollo incluye evidencias de ambos proyectos, pero el análisis de resultados es superficial o faltan capturas. ▪ Tiene 5 referencias APA con errores menores de formato en alguna de ellas. ▪ Redacción generalmente coherente con alguna inconsistencia.	▪ Faltan 2 o 3 secciones obligatorias (ej. sin resumen o sin lecciones aprendidas). ▪ El desarrollo del tema documenta solo uno de los dos proyectos con evidencias. ▪ Menos de 5 referencias o las fuentes no corresponden a libros ni artículos científicos. ▪ Uso inconsistente de normas APA.	▪ El informe existe, pero le faltan más de 3 secciones obligatorias. ▪ El desarrollo del tema no presenta evidencias del código ni capturas de pantalla. ▪ No se utilizan normas APA o el formato es incorrecto en la mayoría de referencias.	▪ El informe no fue entregado o no sigue la estructura solicitada. ▪ No tiene referencias bibliográficas.
VIDEO EN YOUTUBE **Mín. 15 min cámara activa**	**20%**	▪ Duración ≥ 15 minutos; publicado en YouTube con enlace funcional incluido en el informe. ▪ Todos los integrantes participan con cámara activa y explican alguna parte de la solución. ▪ Se muestra la ejecución en vivo de ambos proyectos: código, pruebas de servicio e interfaz. ▪ La explicación es fluida, técnicamente correcta y evidencia comprensión de la arquitectura en capas.	▪ Duración ≥ 15 min; enlace funcional; todos los integrantes participan con cámara. ▪ Se muestra la ejecución de ambos proyectos, pero la explicación de alguna capa es superficial. ▪ La arquitectura en capas se menciona, pero no se explica con profundidad.	▪ Duración entre 10 y 14 minutos, o algún integrante no aparece con cámara activa. ▪ Solo se documenta un proyecto o la demostración de ejecución está ausente. ▪ La explicación es descriptiva sin profundidad técnica.	▪ Duración menor a 10 minutos. ▪ Varios integrantes no participan en cámara ni en la explicación. ▪ El video muestra la interfaz, pero no explica el código ni la arquitectura en capas.	▪ El video no fue entregado o el enlace no funciona. ▪ El contenido del video no corresponde al trabajo desarrollado.

EXPOSICIÓN EN AULA **15 minutos por grupo**	**20%**	▪ Todos los integrantes participan activamente y demuestran dominio del trabajo. ▪ Explican con claridad la arquitectura en capas de ambos proyectos y muestran la ejecución en vivo. ▪ Responden correctamente las preguntas del docente sobre la lógica y el diseño implementado. ▪ El tiempo se distribuye adecuadamente dentro de los 15 minutos asignados.	▪ Todos o la mayoría de integrantes participan con cámara activa. ▪ Cubren ambos proyectos, pero la explicación de alguna capa es superficial. ▪ Responden la mayoría de preguntas del docente con alguna imprecisión técnica. ▪ Manejan el tiempo con pequeños desvíos.	▪ Participan solo uno o dos integrantes; el resto no interviene. ▪ Se expone solo un proyecto o la arquitectura en capas no se explica con claridad. ▪ No logran responder la mayoría de preguntas del docente.	▪ La exposición es muy breve o desorganizada; no se muestra la ejecución del programa. ▪ El grupo no demuestra comprensión de la arquitectura en capas implementada.	▪ El grupo no se presenta a la exposición en la fecha asignada. ▪ El grupo se presenta, pero no expone el trabajo desarrollado.
Nota T1 = Blackboard convierte automáticamente el nivel seleccionado al puntaje según el peso de cada criterio.						